```
  partial class Spaceship : Entity { / ... / }
  partial class Asteriod : Entity { / ... / }
  // in Draw.cs
  abstract partial class Entity { void Draw(); }
  partial class Spaceship : Entity { void Draw() { / ...
/ } }
  partial class Asteriod : Entity { void Draw() { / ...
/ } }
```

Another use of partial classes you'll see is stowing away automatically generated code in a separate partial class definition. This way you can modify an auto-generated class safely in another file. This is how the WinForms designers work. Note that partial methods are also possible. Such methods define their signature in one file and the implementation in another. This is more useful to code generators rather than programmers.

Finally, a powerful extensibility feature of C# are extension methods. Recall that the String class has no Reverse method. The String class itself does not define Reverse; it is instead provided as an extension method (in System.Linq). Here is our version.

```
  public static class MyStringExtensionMethod
  {
      public static string MyRev(this string s)
      {
          string ret = "";
         // Note: I should use StringBuilder here instead
of simple strings
          // to avoid unnecessary allocations.
          foreach (char c in s)
          {
              ret = c + ret;
          }
          return ret;
      }

      public static void Test()
      {
          Console.WriteLine("this is my string".MyRev());
      }
  }
```